

End Semester Exam
CSL7640 - Natural Language Understanding

NOTE:

1. This is a Question-cum-answer sheet. Maximum Points: 80, Total Questions: 36, Total Page Number: 9 + 3 pages of a research paper (including both sides), Total Time: 3 h. **If there is anything not clear in the problems, go ahead with your own assumptions, but state them clearly. No doubts will be entertained during the exam.** Be precise and write the answer in the box provided. Use the answer box spaces judiciously; if you believe the space is less, write only the main steps of the answer. Verbosity will be penalized. Use the other answer sheet for rough work and submit both.

• Name:

Roll Number:

Signature:

Question Number	Topic	Total Marks	Marks Obtained
1-20	Part-A1: Short Answers	40	
21-22	Part-A2: Efficiency in Transformer	10	
23-26	Part-B: BERT Implementation	10	
27-36	Part-C: Paper Understanding	20	
	Total	80	

Anand →

Part-A1: Short Answer Questions Q1-Q20

1. Mention a case where the BLEU score is better than the semantic score-based performance metric for natural language generation. [2 marks]

Where exact match matters, e.g. legal document summarization/translation (Any other example)

2. Which of the following is a drawback of dynamic (contextual) embeddings compared to static embeddings? A. Inability to handle out-of-vocabulary (OOV) words B. Higher inference-time computational cost C. Fixed representation for each word irrespective of context D. Inability to capture polysemy [2 marks]

B

3. What is the primary objective used to train models like GPT? A. Masked language modeling B. Next token prediction C. Sentence similarity prediction D. Named entity recognition E. All of Above [2 marks]

B

4. For a sequence of length $n = 100$ and hidden dimension $d = 64$, what is the FLOPs required to compute QK^T ? [2 marks]

1.27 million

5. Why are encoder-only models generally not preferred for generative tasks? A. They cannot process long sequences B. They lack bidirectional context C. They are not designed for autoregressive token generation D. They cannot be fine-tuned [2 marks]

C

6. Which of the following statements is TRUE? A. Decoder-only models cannot perform translation tasks B. Encoder-decoder models cannot be trained on unlabeled text C. Decoder-only models can leverage large unstructured corpora effectively D. Encoder-only models are best suited for long-form text generation. [2 marks]

C

7. Given word embeddings v_{he} and v_{she} , define the gender direction g . For a gender-neutral word embedding w , derive the formula to remove its gender component.

$$\textcircled{1} \quad \vec{g} = \vec{v}_{he} - \vec{v}_{ske} \quad \checkmark$$

$$\textcircled{2} \quad w' = \vec{w} - \frac{w \cdot \vec{g}}{\vec{g} \cdot \vec{g}} \vec{g}$$

8. A benefit of sinusoidal (hardcoded) positional encodings is that they: A. Removes the need for masking. B. Enable length extrapolation without retraining C. Directly encode relative positions via learned per-head bias D. Remove the need for token embeddings. [2 marks]

B

9. Give one example each for decoder-only and encoder-decoder-based transformer architecture. [2 marks]

decoder-only : GPT
e-decoder : BART, T5

10. List the two main training stages involved in RLHF. [2 marks]

① Reward Modeling
② PPO (Policy Preference Optimization)

11. Which among the following metric needs to be maximized for better retrieval: (a) Precision@k, (b) Recall@k, (c) Mean average precision, (d) MRR (d) NDCG@k, (e) None of these. [2 marks]

(e)

12. A startup built a chatbot using a large language model with a 1 million token context window. They can upload their entire internal knowledge base (e.g., policies, manuals, FAQs) into the prompt itself. However, they still decided to use Retrieval-Augmented Generation (RAG) instead. What might be the two reasons. [2 marks]

→ manual

① Efficiency & Cost (low latency)
② Relevance & signal-to-noise

13. Can any positional embedding be directly applied inside Self-Attention layer? If yes, name the embedding technique. [2 marks]

Yes, Role | Rotary positional embedding
~~inspired~~

14. You are allowed to increase only one of the following model or inference parameters. Which choice would have the least negative impact (decrease latency) and most positive impact (increase latency) on latency? A. Number of layers B. Context length C. Vocabulary size D. Hidden dimension [2 marks]

+ve impact: Context length
 least -ve impact: Vocabulary size

15. You train a Word2Vec model on a dataset containing many rare and morphologically complex words (e.g., unhappiness, replaying, nationalization). Which type of model would better handle such cases, and why?: Skipgram or CBOW or FastText. [2 marks]

① FastText, subword embedding
 ②

16. Why is supervised fine-tuning required for building LLM? [2 marks]

- Align model to task specific behavior
 - Improve output quality and correctness.

17. Give two examples of commercial closed-source LLMs. [2 marks]

ChatGPT, Claude, Gemini

18. What is the major difference between PPO and DPO? [2 marks]

PPO - uses RL, requires separate RL model

19. What are the two major ethical concerns with using LLM agents? [2 marks]

- Amplify bias
 - Sensitive information leak
 - Risk of unauthorized data access

20. Why is KL divergence is used in Reinforcement Learning with Human Feedback (RLHF) e.g., PPO? [2 marks]

KL divergence is used as regularizer to keep the updated model close to previous one.

Part-A2: Efficiency in Transformers

21. [Flash Attention] You are deploying a transformer model on a low-resource device that cannot store all exponential simultaneously due to memory constraints. Therefore, you must compute softmax using an online method maintaining only:

- a running maximum m
- a running normalization term Z

The input arrives sequentially as:

$$x = [2, 4, 1]$$

Process the inputs one by one (left to right) and update m and Z at each step using the online softmax rules.

- (a) What are the final values of m and Z ? [2.5 marks]

$$m = 4, \quad Z = 1 + e^{-2} + e^{-3}$$

- (b) Using these, compute the final softmax probabilities. [2.5 marks]

$$\frac{e^{-2}}{1 + e^{-2} + e^{-3}}, \quad \frac{1}{1 + e^{-2} + e^{-3}}, \quad \frac{e^{-3}}{1 + e^{-2} + e^{-3}} \quad \text{or}$$

$$[0.114, 0.844, 0.042]$$

22. [KV Cache] Consider a decoder-only transformer model with the following specifications:

- Number of layers $L = 12$
- Number of attention heads $h = 8$
- Head dimension $d_h = 64$
- Sequence length generated so far $n = 100$
- Data type: float32 (4 bytes per value)

- (a) Compute the total memory required for the KV cache (keys and values) across all layers. [3 marks]

$$\begin{aligned}
 \text{Total elem} &= 2 \times L \times n \times h \times d_n \\
 &= 2 \times 12 \times 100 \times 8 \times 64 \\
 &= 1228800 \text{ values} \\
 \text{\#Memory size} &= 4915200 \text{ Byte} \\
 &\approx 4.9 \text{ MB}
 \end{aligned}$$

- (b) If one more token is generated (i.e., sequence length becomes $n = 101$), compute the additional memory required for the KV cache. [1 marks]

$$\begin{aligned}
 2 \times L \times h \times d_n &= 12288 \text{ values} \\
 &\approx 49 \text{ KB}
 \end{aligned}$$

- (c) Briefly explain how KV cache reduces computation during autoregressive decoding. [1 marks]

Without KV Cache: ^{for} each token keys and values for all previous tokens. $O(n^2)$
 With KV cache: stores past k, v only current & is computed, $O(n)$

Part-B: BERT Implementation – Q23 to Q26

You have worked on fine-tuning BERT for a Question Answering (QA) task in the assignment. Based on your understanding of the implementation, answer the following:

23. If you were to fine-tune **BERT** for a Named Entity Recognition (NER) task, how would a single training data sample look? Illustrate with an example. [2 marks]

Mohan lives in Delhi
 ↓ ↓ ↓
 [PER] O O [LOC]

24. In the NER setup, where is the prediction head added in the BERT architecture? What is its role? [2 marks]

— Each token, classify the token into

25. What loss function is typically used for training BERT on NER? Briefly justify your choice. [4 marks]

Cross entropy loss
 labels space - [NER, LOC, ...]
 cross entropy

26. Write a short code snippet (3–5 lines) to compute the loss for the NER task. [2 marks]

```
logits = model(input_id).logits
loss_fn = torch.nn.CrossEntropyLoss()
loss = loss_fn(logits.view(-1, num_labels),
               label.view(-1))
```

Part-C: Research Paper Reading and Understanding – Q27 to Q36

You are given a part of a research paper published in ACL 2020 as supplementary pages to this question paper. Read the paper carefully and then answer the following questions:

27. [True/False] For sequence classification tasks, BART uses only the encoder representations (similar to BERT's [CLS] token) and does not involve the decoder during fine-tuning. False [1 mark]
28. [True/False] BART's decoder includes cross-attention over the encoder's final hidden states, enabling it to condition on the entire input during generation. True [1 mark]
29. [True/False] In BART, the encoder operates bidirectionally on corrupted input, while the decoder generates the original sequence autoregressively from left to right. True [1 mark]

30. [True/False] BART's pretraining objective is limited to token masking similar to BERT, and does not allow arbitrary transformations such as sentence shuffling or document rotation. false
[1 mark]

31. [True/False] BART generalizes both BERT and GPT by combining a bidirectional encoder with a left-to-right autoregressive decoder. True [1 mark]

32. [True/False] BART cannot be used for generation tasks. false [1 mark]

33. [True/False] Next token prediction is one of the objectives of the BART architecture. false
[1 mark]

34. Write down the main contributions of this paper in your own words. [6 marks]

- A unified denoising pre-training framework
- Combining E - D
- Systematic study
- Superior performance -

35. Give four reasons why decoder-only models like GPT are often preferred over encoder-decoder models such as BART. [4 marks]

- Simple, scalable and efficient for generation
- highly flexible

36. How would you extend BART to handle multimodal inputs (e.g., image + text)? Briefly describe the architectural changes required. [4 marks]

- Add an Image encoder
- Align image/text emb
- Fuse them in decoder (cross-attention)

BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

Mike Lewis*, Yinhan Liu*, Naman Goyal*, Marjan Ghazvininejad,
Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, Luke Zettlemoyer

Facebook AI

mikelewis@fb.com, yinhan@ai2incubator.com, naman@fb.com

Abstract

We present BART, a denoising autoencoder for pretraining sequence-to-sequence models. BART is trained by (1) corrupting text with an arbitrary noising function, and (2) learning a model to reconstruct the original text. It uses a standard Transformer-based neural machine translation architecture which, despite its simplicity, can be seen as generalizing BERT (due to the bidirectional encoder), GPT (with the left-to-right decoder), and other recent pre-training schemes. We evaluate a number of noising approaches, finding the best performance by both randomly shuffling the order of sentences and using a novel in-filling scheme, where spans of text are replaced with a single mask token. BART is particularly effective when fine tuned for text generation but also works well for comprehension tasks. It matches the performance of RoBERTa on GLUE and SQuAD, and achieves new state-of-the-art results on a range of abstractive dialogue, question answering, and summarization tasks, with gains of up to 3.5 ROUGE. BART also provides a 1.1 BLEU increase over a back-translation system for machine translation, with only target language pretraining. We also replicate other pretraining schemes within the BART framework, to understand their effect on end-task performance.¹

1 Introduction

Self-supervised methods have achieved remarkable success in a wide range of NLP tasks (Mikolov et al., 2013; Peters et al., 2018; Devlin et al., 2019; Joshi et al., 2019; Yang et al., 2019; Liu et al., 2019). The most successful approaches have been variants of masked language models, which are denoising autoencoders that are trained to reconstruct text where a random subset of the words has been masked out. Recent work has shown gains by improving the distribution of

masked tokens (Joshi et al., 2019), the order in which masked tokens are predicted (Yang et al., 2019), and the available context for replacing masked tokens (Dong et al., 2019). However, these methods typically focus on particular types of end tasks (e.g. span prediction, generation, etc.), limiting their applicability.

In this paper, we present BART, which pre-trains a model combining Bidirectional and Auto-Regressive Transformers. BART is a denoising autoencoder built with a sequence-to-sequence model that is applicable to a very wide range of end tasks. Pretraining has two stages (1) text is corrupted with an arbitrary noising function, and (2) a sequence-to-sequence model is learned to reconstruct the original text. BART uses a standard Transformer-based neural machine translation architecture which, despite its simplicity, can be seen as generalizing BERT (due to the bidirectional encoder), GPT (with the left-to-right decoder), and many other more recent pretraining schemes (see Figure 1).

A key advantage of this setup is the noising flexibility; arbitrary transformations can be applied to the original text, including changing its length. We evaluate a number of noising approaches, finding the best performance by both randomly shuffling the order of the original sentences and using a novel in-filling scheme, where arbitrary length spans of text (including zero length) are replaced with a single mask token. This approach generalizes the original word masking and next sentence prediction objectives in BERT by forcing the model to reason more about overall sentence length and make longer range transformations to the input.

BART is particularly effective when fine tuned for text generation but also works well for comprehension tasks. It matches the performance of RoBERTa (Liu et al., 2019) with comparable training resources on GLUE (Wang et al., 2018) and SQuAD (Rajpurkar et al., 2016), and achieves new state-of-the-art results on a range of abstractive dialogue, question answering, and summarization tasks. For example, it improves performance by 3.5 ROUGE over previous work on XSum (Narayan et al., 2018).

BART also opens up new ways of thinking about fine tuning. We present a new scheme for machine translation where a BART model is stacked above a few additional transformer layers. These layers are trained

¹Code and pre-trained models for BART are available at <https://github.com/pytorch/fairseq> and <https://huggingface.co/transformers>

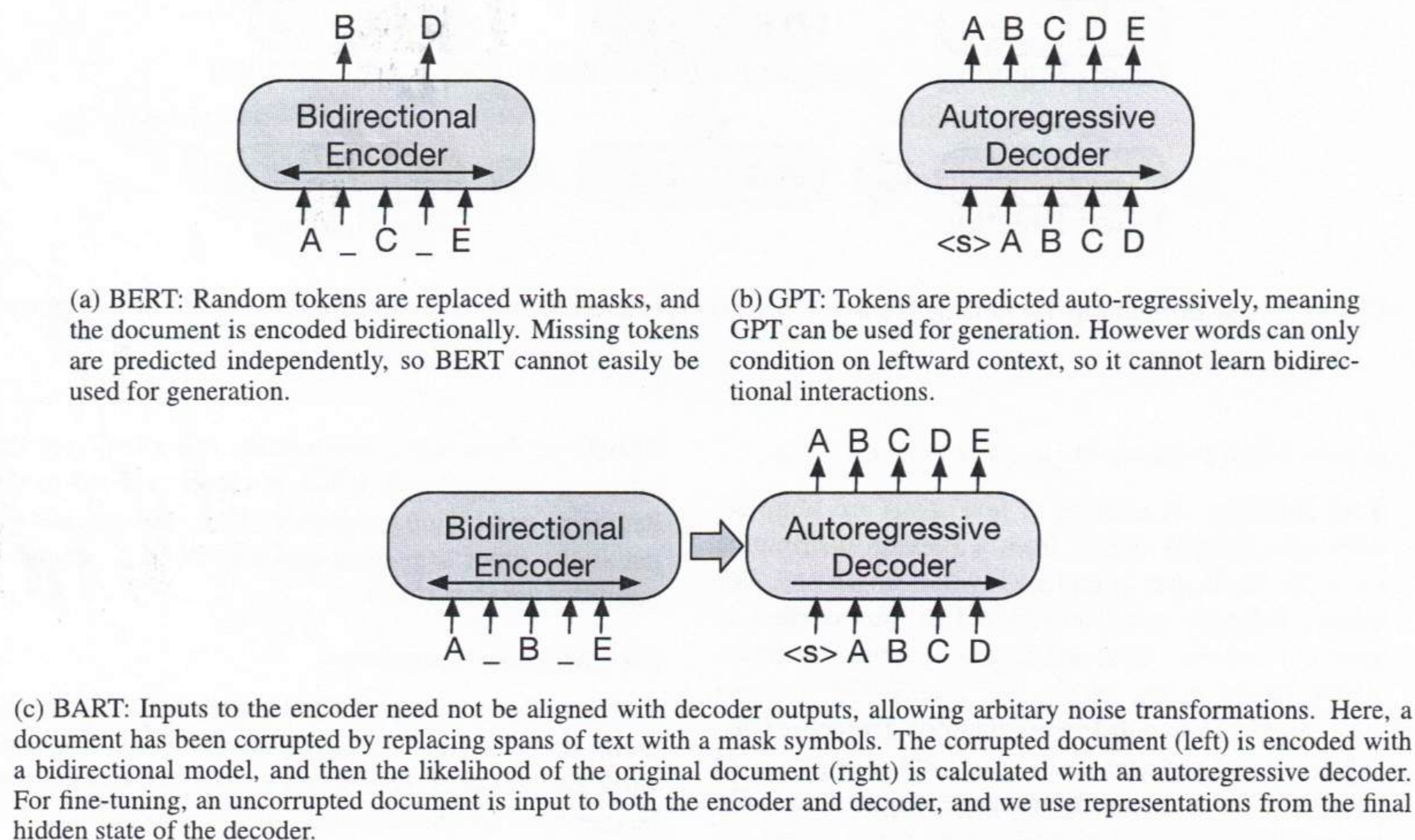


Figure 1: A schematic comparison of BART with BERT (Devlin et al., 2019) and GPT (Radford et al., 2018).

to essentially translate the foreign language to noised English, by propagation through BART, thereby using BART as a pre-trained target-side language model. This approach improves performance over a strong back-translation MT baseline by 1.1 BLEU on the WMT Romanian-English benchmark.

To better understand these effects, we also report an ablation analysis that replicates other recently proposed training objectives. This study allows us to carefully control for a number of factors, including data and optimization parameters, which have been shown to be as important for overall performance as the selection of training objectives (Liu et al., 2019). We find that BART exhibits the most consistently strong performance across the full range of tasks we consider.

2 Model

BART is a denoising autoencoder that maps a corrupted document to the original document it was derived from. It is implemented as a sequence-to-sequence model with a bidirectional encoder over corrupted text and a left-to-right autoregressive decoder. For pre-training, we optimize the negative log likelihood of the original document.

2.1 Architecture

BART uses the standard sequence-to-sequence Transformer architecture from (Vaswani et al., 2017), except, following GPT, that we modify ReLU activation functions to GeLUs (Hendrycks & Gimpel, 2016) and initialise parameters from $\mathcal{N}(0, 0.02)$. For our

base model, we use 6 layers in the encoder and decoder, and for our large model we use 12 layers in each. The architecture is closely related to that used in BERT, with the following differences: (1) each layer of the decoder additionally performs cross-attention over the final hidden layer of the encoder (as in the transformer sequence-to-sequence model); and (2) BERT uses an additional feed-forward network before word-prediction, which BART does not. In total, BART contains roughly 10% more parameters than the equivalently sized BERT model.

2.2 Pre-training BART

BART is trained by corrupting documents and then optimizing a reconstruction loss—the cross-entropy between the decoder’s output and the original document. Unlike existing denoising autoencoders, which are tailored to specific noising schemes, BART allows us to apply *any* type of document corruption. In the extreme case, where all information about the source is lost, BART is equivalent to a language model.

We experiment with several previously proposed and novel transformations, but we believe there is a significant potential for development of other new alternatives. The transformations we used are summarized below, and examples are shown in Figure 2.

Token Masking Following BERT (Devlin et al., 2019), random tokens are sampled and replaced with [MASK] elements.

Token Deletion Random tokens are deleted from the input. In contrast to token masking, the model must

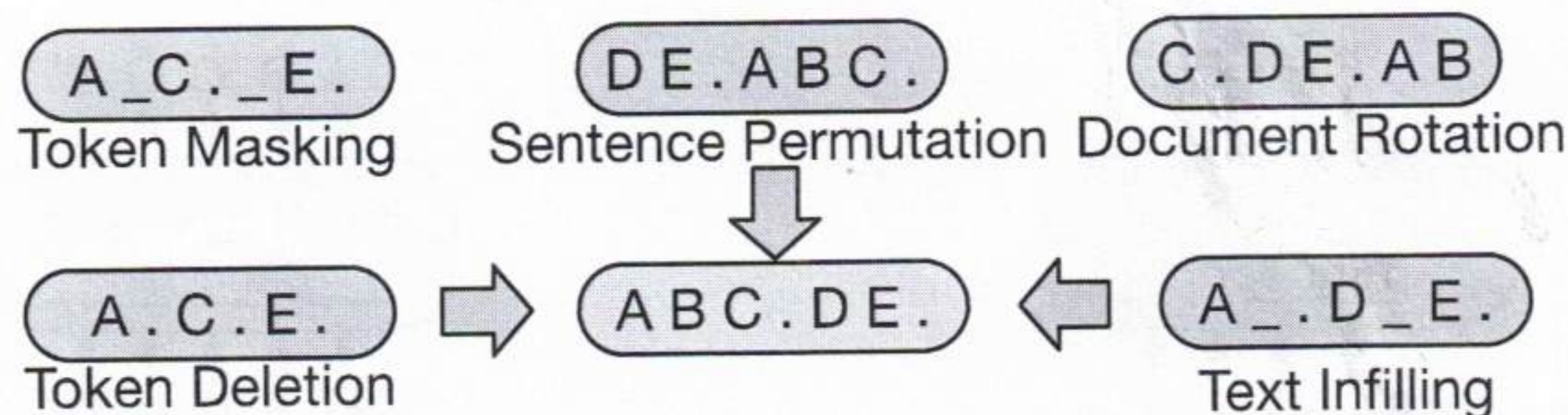


Figure 2: Transformations for noising the input that we experiment with. These transformations can be composed.

decide which positions are missing inputs.

Text Infilling A number of text spans are sampled, with span lengths drawn from a Poisson distribution ($\lambda = 3$). Each span is replaced with a *single* [MASK] token. 0-length spans correspond to the insertion of [MASK] tokens. Text infilling is inspired by SpanBERT (Joshi et al., 2019), but SpanBERT samples span lengths from a different (clamped geometric) distribution, and replaces each span with a sequence of [MASK] tokens of exactly the same length. Text infilling teaches the model to predict how many tokens are missing from a span.

Sentence Permutation A document is divided into sentences based on full stops, and these sentences are shuffled in a random order.

Document Rotation A token is chosen uniformly at random, and the document is rotated so that it begins with that token. This task trains the model to identify the start of the document.

3 Fine-tuning BART

The representations produced by BART can be used in several ways for downstream applications.

3.1 Sequence Classification Tasks

For sequence classification tasks, the same input is fed into the encoder and decoder, and the final hidden state of the final decoder token is fed into new multi-class linear classifier. This approach is related to the CLS token in BERT; however we add the additional token to the *end* so that representation for the token in the decoder can attend to decoder states from the complete input (Figure 3a).

3.2 Token Classification Tasks

For token classification tasks, such as answer endpoint classification for SQuAD, we feed the complete document into the encoder and decoder, and use the top hidden state of the decoder as a representation for each word. This representation is used to classify the token.

3.3 Sequence Generation Tasks

Because BART has an autoregressive decoder, it can be directly fine tuned for sequence generation tasks such as abstractive question answering and summarization.

In both of these tasks, information is copied from the input but manipulated, which is closely related to the denoising pre-training objective. Here, the encoder input is the input sequence, and the decoder generates outputs autoregressively.

3.4 Machine Translation

We also explore using BART to improve machine translation decoders for translating into English. Previous work Edunov et al. (2019) has shown that models can be improved by incorporating pre-trained encoders, but gains from using pre-trained language models in decoders have been limited. We show that it is possible to use the entire BART model (both encoder and decoder) as a single pretrained decoder for machine translation, by adding a new set of encoder parameters that are learned from bitext (see Figure 3b).

More precisely, we replace BART’s encoder embedding layer with a new randomly initialized encoder. The model is trained end-to-end, which trains the new encoder to map foreign words into an input that BART can de-noise to English. The new encoder can use a separate vocabulary from the original BART model.

We train the source encoder in two steps, in both cases backpropagating the cross-entropy loss from the output of the BART model. In the first step, we freeze most of BART parameters and only update the randomly initialized source encoder, the BART positional embeddings, and the self-attention input projection matrix of BART’s encoder first layer. In the second step, we train all model parameters for a small number of iterations.

4 Comparing Pre-training Objectives

BART supports a much wider range of noising schemes during pre-training than previous work. We compare a range of options using base-size models (6 encoder and 6 decoder layers, with a hidden size of 768), evaluated on a representative subset of the tasks we will consider for the full large scale experiments in §5.

4.1 Comparison Objectives

While many pre-training objectives have been proposed, fair comparisons between these have been difficult to perform, at least in part due to differences in training data, training resources, architectural differ-